

Duy trì tính liên thông của rừng: cây động

Chen Shouyuan

2006

Nguồn gốc: Maintaining forest connectivity: dynamic trees

IOI China candidate team papers / 2006 / Chen Shouyuan / Chen Shouyuan.doc

1 Giới thiệu

Bài viết giới thiệu một cấu trúc dữ liệu gọi là **cây động**. Cấu trúc này có thể duy trì một rừng có trọng số, hỗ trợ thao tác link để nối hai cây, và thao tác cut để xóa một cạnh làm một cây tách thành hai cây. Nó có rất nhiều ứng dụng trong tối ưu hóa mạng.

2 Các thao tác cơ bản

Cây động cần hỗ trợ các thao tác sau:

- $\text{Parent}(v)$: trả về cha của v , nếu v là gốc thì trả về null.
- $\text{Root}(v)$: trả về gốc của cây chứa v .
- $\text{Cost}(v)$: trả về chi phí cạnh $(v, \text{parent}(v))$, giả sử v không phải gốc.
- $\text{Mincost}(v)$: trả về cạnh có trọng số nhỏ nhất trên đường từ $\text{root}(v)$ đến v .
- $\text{Update}(v, x)$: cộng x vào trọng số mọi cạnh trên đường từ $\text{root}(v)$ đến v .
- $\text{Link}(v, w, x)$: nối cây có gốc v vào đỉnh w , với chi phí cạnh (v, w) là x .
- $\text{Cut}(v)$: xóa cạnh $(v, \text{parent}(v))$, tách cây thành hai cây.
- $\text{Evert}(v)$: đổi gốc, đặt v làm gốc và đảo chiều mọi cạnh trên đường từ v đến gốc cũ.

Thông qua hai lần Update , ta có thể sửa chi phí một cạnh. Mincost cũng có thể thay bằng Maxcost .

Giả sử ban đầu có n đỉnh rời rạc, sau đó thực hiện m thao tác thuộc các loại trên. Cách thô là lưu $\text{dparent}(v)$ và $\text{dcost}(v)$, lần lượt là cha của v và chi phí cạnh nối tới cha. Khi đó parent , cost , link , cut có thể làm trong $O(1)$, nhưng root , mincost , evert , update phụ thuộc vào độ sâu của cây, xấu nhất là $O(n)$.

Thuật toán trong bài không thao tác trực tiếp trên toàn bộ cây. Nó thực hiện m thao tác trong $O(m \log n)$ thời gian khâu hao.

3 Cạnh thực, cạnh ảo và đường

Chia các cạnh trong cây thành hai loại: **cạnh thực** và **cạnh ảo**. Từ mỗi đỉnh đi xuống con, nhiều nhất có một cạnh thực. Một **đường** gồm một số cạnh thực liên thông theo chiều từ dưới lên. Các cạnh còn lại

là cạnh ảo; chúng được lưu bằng $\text{dparent}(v)$ và $\text{dcost}(v)$. Cạnh ảo có thể được chuyển thành cạnh thực trong những điều kiện thích hợp. Nếu $\text{tail}(P)$ là gốc cây thì $\text{dparent}(P) = \text{null}$.

Nhờ thao tác trên các đường chỉ gồm cạnh thực, ta có thể thực hiện thao tác cây động. Trước hết giả sử đã có các thao tác đường sau.

4 Các thao tác trên đường

- $\text{Path}(v)$: trả về đường chứa v .
- $\text{Head}(p)$: trả về đỉnh đầu của đường p .
- $\text{Tail}(p)$: trả về đỉnh cuối của đường p .
- $\text{Before}(v)$: trả về đỉnh trước v trong đường.
- $\text{After}(v)$: trả về đỉnh sau v trong đường.
- $\text{Pcost}(v)$: trả về chi phí cạnh $(v, \text{after}(v))$.
- $\text{Pmincost}(p)$: trả về cạnh có chi phí nhỏ nhất trong đường p .
- $\text{Pupdate}(p, x)$: cộng x vào chi phí mọi cạnh trong đường p .
- $\text{Reverse}(p)$: đảo chiều mọi cạnh trong đường p .
- $\text{Concatenate}(p, q, x)$: thêm cạnh $(\text{tail}(p), \text{head}(q))$ có chi phí x , rồi gộp hai đường p, q .
- $\text{Split}(v)$: xóa hai cạnh kề v trong $\text{path}(v)$, tách đường thành ba phần và trả về p, q, x, y , trong đó p là đoạn từ đầu đường đến $\text{before}(v)$, q là đoạn từ $\text{after}(v)$ đến cuối đường, x là chi phí cạnh $(\text{before}(v), v)$, và y là chi phí cạnh $(v, \text{after}(v))$.

Nếu v là đỉnh đầu thì $p = \text{null}$; nếu v là đỉnh cuối thì $q = \text{null}$.

5 splice và expose

Hai thao tác sau dùng để duy trì cạnh thực, cạnh ảo, rồi từ đó cài đặt các thao tác cơ bản.

Splice(p:path). Tác dụng của `splice` là kéo đường p lên gần gốc hơn. Cách làm: biến cạnh ảo $(\text{tail}(P), \text{dparent}(P))$ thành cạnh thực. Để duy trì tính chất “mỗi đỉnh chỉ có nhiều nhất một cạnh thực đi xuống con”, cạnh thực cũ xuất phát từ $\text{dparent}(P)$ được chuyển thành cạnh ảo.

```
Function splice(p:Path);
Begin
  U:=dparent(tail(P));
  [q,r,x,y]:=split(u);
  If q<>nil Then Begin
    dparent(tail(q)):=v;
    dcost(tail(q)):=x;
  End;
  P:=concatenate(p,path(P),dcost(tail(P)));
  If r=nil Then return p
  Else return concatenate(p,r,y);
End;
```

Expose(v:vertex). Tác dụng của expose là biến mọi cạnh trên đường từ v đến $\text{root}(v)$ thành cạnh thực. Cách làm là liên tục gọi splice cho đến khi gặp gốc.

```

Function expose(v:vertex);
Begin
  [q,r,x,y]:=split(v);
  If q<>nil Then Begin
    Dparent(tail(q)):=v;
    Dcost(tail(q)):=x;
  End;
  If r=nil Then p:=path(V)
  Else p:=concatenate(path(V),r,y);
  While dparent(v)<>nil do p:=splice(p);
  Return p;
End;

```

6 Cài đặt các thao tác cây động

Với splice, expose và các thao tác đường, tám thao tác cơ bản của cây động được cài đặt như sau.

```

Function Parent(v:vertex)
Begin
  If v=tail(path(v)) Then Return Dparent(v)
  Else Return after(v)
End;

```

```

Function cost(v:vertex)
Begin
  If v=tail(path(v)) Return dcost(v)
  Else Return Pcost(v)
End;

```

```

Function root(v:vertex);
Begin
  Return tail(expose(v));
End;

```

```

Function Mincost(v:vertex);
Begin
  Return Pmincost(expose(v));
End;

```

```

Procedure Update(v:vertex;x:real);
Begin
  Pupdate(expose(v),x);
End;

```

```

Procedure Link(v,w:vertex;x:real);

```

```

Begin
  Concatenate(path(v), expose(w), x);
End;

Function Cut(v:vertex);
Var
  p,q:path;
  x,y:real;
Begin
  Expose(v);
  [p,q,x,y]:=split(v);
  Dparent(v):=nil;
  Return y;
End;

Procedure Evert(v);
Begin
  Reverse(expose(v));
  Dparent(v):=nil;
End;

```

Đến đây, bài toán trên cây đã được chuyển thành bài toán trên các đường.

7 Cài đặt cấu trúc đường

Có thể dùng cây splay để lưu mỗi đường. Các đỉnh trên đường được sắp theo khoảng cách tới đỉnh đầu đường.

Đảo đường. Mỗi nút lưu một cờ *reverse*. Nếu cờ này bật, cây con gốc ở nút đó cần bị đảo, tức là con trái và con phải của mọi nút trong cây con cần đổi chỗ. Khi truy cập nút v , nếu cờ *reverse* bật, ta đổi hai con, tắt cờ ở v , rồi truyền cờ xuống các con bằng phép lấy phủ định. Thao tác Reverse chỉ cần đổi cờ ở gốc.

Cộng trọng số trên đường. Với mỗi nút v , lưu *ecost* và *change*. Giá trị *ecost* là chi phí cạnh ($v, \text{after}(v)$); *change* là lượng hiệu chỉnh chung của các cạnh trong cây con gốc v . Khi truy cập v , cộng *change* vào *ecost*, đặt *change* = 0, rồi truyền lượng hiệu chỉnh xuống các con. Mỗi nút còn lưu *emin*, là chi phí nhỏ nhất trong cây con của nó. Thao tác Pupdate chỉ cần sửa *change* ở gốc.

Các thao tác Path, Tail, Head, Before, After, Concatenate và Split đều có thể cài bằng các thao tác splay quen thuộc, đồng thời duy trì các giá trị *reverse*, *change*, *ecost* và *emin*.

8 Phân tích độ phức tạp

Mỗi thao tác cây động dùng một lần expose. Ta xét số lần splice trong expose. Với cạnh $v \rightarrow w$ trong cây động, nếu số đỉnh trong cây con của v lớn hơn một nửa số đỉnh trong cây con của w , gọi đó là cạnh loại A; ngược lại là cạnh loại B. Mỗi đỉnh nhiều nhất có một cạnh loại A đi ra, nên trên mọi đường có nhiều nhất $O(\log n)$ cạnh loại B.

Gọi p là số cạnh ảo loại B. Trong một lần splice, nếu thêm một cạnh ảo loại B vào đường thì p tăng 1; tình huống này chỉ xảy ra $O(\log n)$ lần. Nếu thêm một cạnh ảo loại A vào đường thì p giảm 1; việc này có thể xảy ra nhiều lần, nhưng chi phí được trả bằng thể năng p , vốn luôn không âm. Vì vậy trung bình mỗi expose thực hiện $O(\log n)$ thao tác đường.

Nếu cài đường bằng cây AVL hoặc đỏ-đen, mỗi thao tác đường mất $O(\log n)$, cho tổng $O(\log^2 n)$. Với cây splay, phân tích khấu hao tốt hơn và đạt $O(\log n)$ cho mỗi thao tác cây động.

Ý tưởng thể năng là: với mỗi đỉnh i , gọi $s(i)$ là số đỉnh trong cây con gốc i của cây động, và $r(i) = \log_2 s(i)$. Thể năng của một cây động là tổng các $r(i)$. Tính chất của splay cho biết chi phí khấu hao của một lần splay(x) không vượt quá

$$3(r(t) - r(x)) + 1,$$

với t là gốc cây splay. Cộng dồn các thao tác trong expose cho thấy chi phí khấu hao của expose là $O(\log n)$. Các thao tác link và cut có thể làm thay đổi thể năng cây động, nhưng mức thay đổi vẫn là bậc logarit.

9 Ứng dụng

- **Tổ tiên chung gần nhất.** Để hỏi LCA của v, w , trước hết thực hiện expose(v), rồi thực hiện expose(w). Khi chạy expose(w), ghi lại điểm gần nhất đã được truy cập trong lần expose trước; đó chính là LCA. Mỗi truy vấn mất $O(\log n)$.
- **Hợp nhất và tách tập.** Có thể hỗ trợ Union và Find, đồng thời hỗ trợ tách theo một số cách, với mỗi thao tác $O(\log n)$.
- **Luồng cực đại.** Cây động có thể tối ưu thuật toán tăng đường ngắn nhất, làm mỗi lần tăng còn $O(m \log n)$, tổng thời gian $O(mn \log n)$.
- **Cây khung nhỏ nhất.** Cây động có nhiều ứng dụng trong thuật toán tăng dần cho cây khung nhỏ nhất, cây khung bị ràng buộc bậc và nhiều biến thể khác.

10 Ghi chú về cài đặt

Nguồn gốc còn trình bày một biến thể cài đặt dùng **cây ảo**. Mỗi đỉnh của cây ảo tương ứng một-một với đỉnh của cây động. Mỗi đỉnh nhiều nhất có hai cạnh thực, gọi là con trái và con phải; các đỉnh khác nối bằng cạnh ảo. Mỗi đỉnh lưu cha $p(x)$, con trái $\text{left}(x)$, con phải $\text{right}(x)$, chi phí $\text{cost}(x)$, chi phí nhỏ nhất trong cây con $\text{mincost}(x)$, cờ reverse, cùng hai lượng dcost và dmin . Với gốc cây thực:

$$\text{dcost}(x) = \text{cost}(x),$$

còn nếu không phải gốc:

$$\text{dcost}(x) = \text{cost}(x) - \text{cost}(p(x)).$$

Ngoài ra

$$\text{dmin}(x) = \text{cost}(x) - \text{mincost}(x).$$

Biến thể này cùng bản chất với thuật toán đã mô tả, nên độ phức tạp khấu hao vẫn là $O(\log n)$ cho mỗi thao tác.

Tài liệu tham khảo

- Sleator and Tarjan, *A data structure for dynamic trees*.
- Sleator and Tarjan, *Self-adjusting binary search trees*.
- Georgiadis, Tarjan, Werneck, *Design of data structures for mergeable trees*.
- Ravindra K. Ahuja, Thomas L. Magnanti, James B. Orlin, *Network Flows: Theory, Algorithms, and Applications*.